JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Generative AI

Autoregressive and Classical Models

Outline

- Module Learning Objectives
- Gaussian Mixture Models (GMMs)
- Hidden Markov Models (HMMs)
- Bayesian Networks
- Autoregressive Time Series Models
- Recurrent Neural Networks

Module Learning Objectives

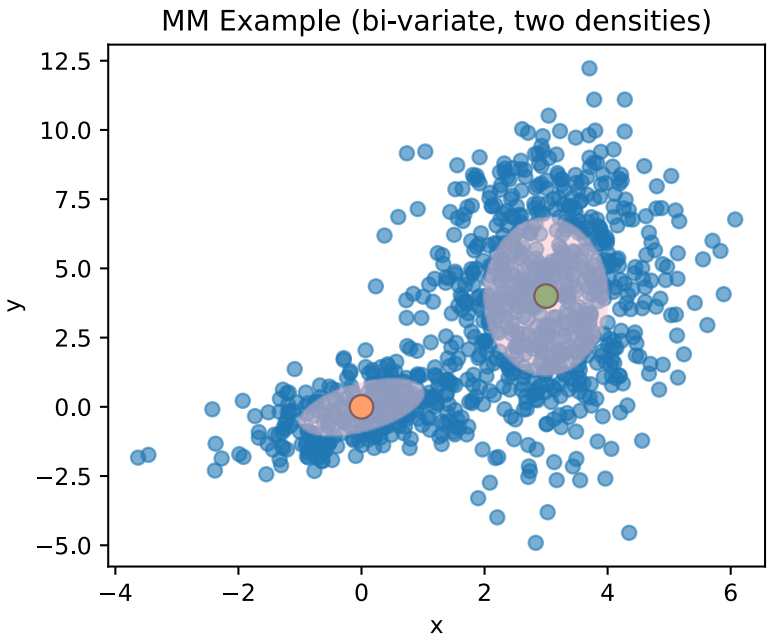
- Study fundamental principles and applications of autoregressive methods
- Comprehend the structure and functioning of Gaussian Mixture Models, focusing on the Expectation-Maximization (EM) algorithm, latent variables, and their use in density estimation, clustering, and anomaly detection
- Grasp the concepts underlying Hidden Markov Models, including transition probabilities, emission probabilities, states, and the Viterbi algorithm
- Understand the principles and construction of Bayesian Networks, including directed acyclic graphs (DAGs), conditional independence, inference, and learning
- Learn classical machine learning models
  - Explain Autoregressive (AR) algorithms and their role in time series forecasting

Classical Generative AI Methods

- Gaussian Mixture Models (GMMs)
  - Image compression and clustering, speaker identification, anomaly detection
- Hidden Markov Models (HMMs)
  - NLP, Speech synthesis and recognition, sequence prediction in bioinformatics
- Latent Dirichlet Allocation (LDA) is a generative probabilistic model
  - Topic discovery in large corpora, document classification, recommender systems
- Variational Autoencoders (VAEs)
  - Image generation, anomaly detection, synthetic generation to augment data
- Restricted Boltzmann Machines (RBMs) is a generative stochastic neural network
  - Collaborative filtering, dimensionality reduction, feature extraction

Gaussian Mixture Models

- Versatile method in classical generative AI for modeling complex, multidimensional data
- Capture data's underlying statistical properties
- Generate new data points based on statistical distributions



Define a Gaussian Mixture Model

- Assume data is generated as a mixture of Gaussian distributions
  - Each Gaussian distribution in the mixture is defined by its mean vector μ and covariance matrix Σ
  - The density of a GMM for a d-dimensional data point x is given by

$$p(x) = \sum_{k=1}^{K} \pi_k N(x \mid \mu_k, \Sigma_k)$$

  - K is the number of Gaussian components
  - π_k are the mixing coefficients with $\sum_{k=1}^{K} \pi_k = 1$ and $0 \leq \pi_k \leq 1$
  - $N(x \mid \mu_k, \Sigma_k)$ is the multivariate Gaussian distribution with μ_k and Σ_k

$$N(x \mid \mu_k, \Sigma_k) = \frac{1}{(2\pi)^{d/2} |\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_k)^{\mathrm{T}} \Sigma_k^{-1} (x - \mu_k)\right)$$

Expectation-Maximization Algorithm

- **E-Step (Expectation)**
- Expected value of the latent variables
- Calculate the **responsibilities** γ_{ik} which are the probabilities that data point x_i belongs to Gaussian component k

$$\gamma_{ik} = \frac{\pi_k N(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j N(x_i \mid \mu_j, \Sigma_j)}$$

- **M-Step (Maximization)**
- Update the parameters using the responsibilities

$$\pi_k = \frac{1}{N} \sum_{i=1}^{N} \gamma_{ik}$$

$$\mu_k = \frac{\sum_{i=1}^{N} \gamma_{ik} x_i}{\sum_{i=1}^{N} \gamma_{ik}}$$

$$\Sigma_k = \frac{\sum_{i=1}^{N} \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^T}{\sum_{i=1}^{N} \gamma_{ik}}$$

- **Alternate** between E and M steps until the parameters converge

Types of GMMs

Diagonal GMM

$$\sum_k = \mathrm{diag}(\sigma_{k1}^2, \sigma_{k2}^2, \dots, \sigma_{kd}^2)$$

- The covariance matrices Σ_k are diagonal, the off-diagonal elements are zero
- Assumes features are uncorrelated, but each feature can have a different variance

Tied GMM

$$\sum_k = \sum$$

- All Gaussian components share the same covariance matrix Σ_k

Spherical GMM

$$\sum_k = \sigma_k^2 \mathbf{I}$$

- The covariance matrices Σ_k are proportional to the identity matrix, all features have the same variance and are uncorrelated

Full (Unrestricted)

$$\sum_k = \begin{bmatrix} \sigma_{k_{11}} & \sigma_{k_{12}} & \cdots & \sigma_{k_{1d}} \\ \sigma_{k_{21}} & \sigma_{k_{22}} & \cdots & \sigma_{k_{2d}} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{k_{d1}} & \sigma_{k_{d2}} & \cdots & \sigma_{k_{dd}} \end{bmatrix}$$

- The covariance matrices Σ_k are full matrices, they can have any value and capture correlations between features

GMMs and Generative AI

- **Data Modeling** mixture model complex, multidimensional data distributions, making them useful in Generative AI for understanding the underlying structure of data
- **Data Generation** MMs can generate new data points by sampling from the underlying distributions they model, which is a key aspect of generative processes
- **Unsupervised Learning** MMs, natural unsupervised learning methods, help identify and generate clusters of similar data, can do clustering-based data synthesis
- **Anomaly Detection** MMs can identify and generate data points that are likely or unlikely based on the modeled distribution, can generate anomalous data points
- **Complex Models** MMs provide a probabilistic framework that can be extended or integrated into more complex generative models, such as VAEs or GANs

Hidden Markov Models

- A statistical model representing a system with **hidden states** through observable sequences
  - A set of hidden states $S = \{s_1, s_2, \dots, s_N\}$
  - A set of observable symbols $O = \{o_1, o_2, \dots, o_M\}$
  - A state transition probability matrix $A = [a_{ij}]$ where $a_{ij} = P(s_j \mid s_i)$
  - An observation probability matrix $B = [b_j(k)]$ where $b_j(k) = P(o_k \mid s_j)$
  - An initial state distribution $\pi = [\pi_i]$ where $\pi_i = P(s_i)$

Hidden Markov Model

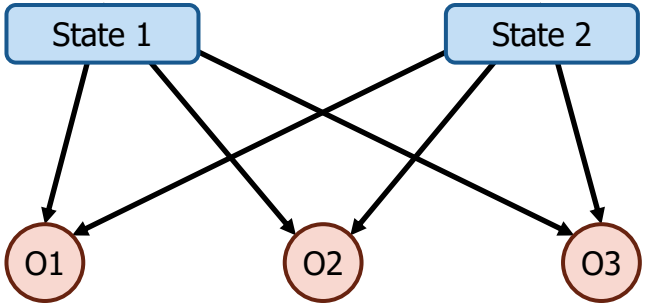
- The probability of observing a seq $O = \langle o_1 o_2 \dots o_T \rangle$ given the model $\lambda = (A, B, \pi)$

$$P(O \mid \lambda) = \sum_{all\ state\ sequences\ Q} P(O \mid Q, \lambda)\, P(Q \mid \lambda)$$

- where $Q = \{q_1, q_2, \dots, q_N\}$ is a set of N states
- The forward algorithm computes this efficiently $\alpha_t(j) = P(o_1 o_2 \dots o_t, q_t = s_j \mid \lambda)$
  - initialized by $\alpha_1(j) = \pi_j b_j(o_1)$
  - updated by $\alpha_{t+1}(j) = \left(\sum_{i=1}^{N} \alpha_t(i) a_{ij}\right) b_j(o_{t+1})$
- The backward algorithm is used to compute $\beta_t(i) = P(o_1 o_2 \dots o_T \mid q_t = s_i, \lambda)$
  - initialized by $\beta_T(i) = 1$
  - updated by $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)$

Sequence Generation

- To generate a new sequence, use the Viterbi algorithm to find the most probable sequence of states and then sample from the observations
- **Example**
  - Transition probabilities
    - P(S1→S1)=0.7, P(S1→S2)=0.3
    - P(S2→S1)=0.4, P(S2→S2)=0.6
  - Emission probabilities
    - B(S1→O1)=0.5, B(S1→O2)=0.4, B(S1→O3)=0.1
    - B(S2→O1)=0.1, B(S2→O2)=0.3, B(S2→O3)=0.6
  - Initial probabilities, π(S1)=0.6, π(S2)=0.4



Viterbi Algorithm

- **Initialization**
  - for each state s_i $\delta_1(i) = \pi_i b_i(o_1)$, where
  - $\delta_1(i)$ is probability of the most probable path ending in state s_i at time t
  - π_i is the initial probability of state s_i $q_T^* = \underset{i}{\operatorname{argmax}}\, \delta_T(i)$
  - $b_i(o_1)$ is the probability of observing o_1 in state s_i
- **Recursion**
  - for each time step t from 2 to T, and for each state s_j
  - $\delta_t(j) = \max_i \left[\delta_{t-1}(i) a_{ij}\right] b_j(o_t)$
  - $\psi_t(j) = \underset{i}{\operatorname{argmax}} \left[\delta_{t-1}(i) a_{ij}\right]$

Viterbi Algorithm

- **Termination**
  - Find the highest probability of the final observation being generated by any state $\mathrm{P}^* = \max_i \delta_T(i)$
  - Identify the state that corresponds to this maximum probability $q_T^* = \operatorname{argmax}_i \delta_T(i)$

- **Path backtracking**
  - Backtrack through ψ matrix to retrieve the most probable path $q_t^* = \psi_{t+1}(q_{t+1}^*)$
  - Start from q_T^* and move backward to get the entire sequence $q_1^*, q_2^*, \ldots, q_T^*$

Pseudocode

Input: Observations $O = \{o_1, o_2, \dots, o_T\}$

Output: Most probable state sequence $\{q_1^*, q_2^*, \dots, q_T^*\}$

1. **Initialization:**

$$\delta_1(i) = \pi_i b_i(o_1)$$
$$\psi_1(i) = 0$$

2. **Recursion:**

For $t = 2$ to T:

$$\delta_t(j) = \max_i \left[\delta_{t-1}(i) a_{ij}\right] b_j(o_t)$$
$$\psi_t(j) = \arg\max_i \left[\delta_{t-1}(i) a_{ij}\right]$$

3. **Termination:**

$$P^* = \max_i \delta_T(i)$$
$$q_T^* = \arg\max_i \delta_T(i)$$

4. **Path Backtracking:**

For $t = T - 1$ to 1:

$$q_t^* = \psi_{t+1}(q_{t+1}^*)$$

Decoding Example

Input: Observations $O = \{P_2, P_3, P_1, P_4\}$

Output: Most probable state sequence $\{q_1^*, q_2^*, q_3^*, q_4^*\}$

1. **Initialization:**

$$\delta_1(i) = \pi_i b_i(P_2)$$
$$\psi_1(i) = 0$$

2. **Recursion:**

For $t = 2$ to 4:

$$\delta_t(j) = \max_i \left[\delta_{t-1}(i) a_{ij}\right] b_j(o_t)$$
$$\psi_t(j) = \arg\max_i \left[\delta_{t-1}(i) a_{ij}\right]$$

3. **Termination:**

$$P^* = \max_i \delta_4(i)$$
$$q_4^* = \arg\max_i \delta_4(i)$$

4. **Path Backtracking:**

For $t = 3$ to 1:

$$q_t^* = \psi_{t+1}(q_{t+1}^*)$$

HMMs and Generative AI

- **Data Generation** HMMs generate sequences by sampling from learned probability distributions, generating new data similar to the training data
- **Probabilistic Framework** HMMs model joint probability distributions of sequences and hidden states, so it can learn and replicate data patterns
- **Latent Variables** HMMs use hidden states to represent unobservable processes, a key concept in models like VAEs and GANs, relying on latent spaces for generation
- **Sequential Data** Modeling HMMs are designed to handle temporal dependencies in data, helping tasks like language modeling and time-series prediction
- **Inference Techniques** HMMs utilize the Expectation-Maximization (EM) algorithm for parameter estimation, a common technique using models with hidden variables
- **Advanced Models** HMMs have inspired the creation of more complex generative models (e.g., RNNs, VAEs), to handle diverse and high-dimensional data

Bayesian Networks

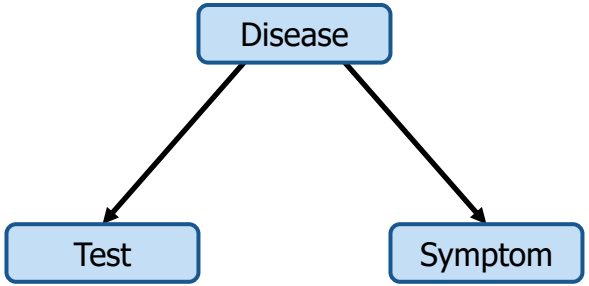
- A Bayesian Network is a directed acyclic graph (DAG)
  - each node represents a random variable
  - each edge represents a conditional dependency between the connected nodes
- The joint probability distribution of the variables can be decomposed into a product of conditional probabilities
- Let $X = \{X_1, X_2, \dots, X_n\}$ be a set of random variables
- The joint probability distribution of X in a Bayesian network is given by

$$P(X) = \prod_{i=1}^{n} P(X_i \mid \mathrm{Parents}(X_i))$$

- where $\mathrm{Parents}(X_i)$ denotes the set of parent nodes of X_i

Medical Diagnosis Example

- Consider a simple medical diagnosis problem with the variables
- **D** Disease (True/False)
- **T** Test Result (Positive/Negative)
- **S** Symptom (Present/Absent)



Medical Diagnosis Example

- Find P(D=True | T=Pos,S=Pres)
- $= \frac{P(T=Pos,S=Pres \mid D=True)P(D=True)}{P(T=Pos,S=Pres)}$
- Compute P(T=Pos,S=Pres)
- =P(T=Pos,S=Pres | D=T)P(D=T)+ P(T=Pos,S=Pres | D=F)P(D=F)
- P(T=Pos,S=Pres | D=F)=0.2×0.1=0.02
- =0.72×0.01+0.02×0.99=0.027
- P(T=Pos,S=Pres)=0.72×0.01+0.02×0.99= 0.0027
- P(D=True | T=Pos,S=Pres)=0.72×0.01/0.027=0.267

D	P(D)
True	0.01
False	0.99

Prior Probability of Disease

D	T	P(T|D)
True	Pos	0.9
True	Neg	0.1
False	Pos	0.2
False	Neg	0.8

D	S	P(S|D)
True	Present	0.8
True	Absent	0.2
False	Present	0.1
False	Absent	0.9

Bayesian Networks and Generative AI

- **Probabilistic Framework** Bayesian Networks model joint probability distributions, crucial for capturing and generating realistic data patterns in Generative AI
- **Conditional Dependencies** Explicitly represents how variables influence each other, ensuring generated data maintains the original structure and relationships
- **Latent Variables** Incorporates hidden factors that drive observed data, enabling the modeling of complex, underlying processes in Generative AI
- **Efficient Inference** Facilitates quick and accurate inference of unknown variables, aiding in tasks like prediction, data imputation, and decision-making
- **Data Generation** Allows for the simulation of new data by sampling from learned distributions, ensuring consistency with the modeled dependencies
- **Advanced Models** Serves as a basis for developing more sophisticated generative models, extending to dynamic, time-dependent, and multidimensional data scenarios

Time Series Analysis

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Time Series

- In classical ML, a single data point used $x \in \mathbb{R}^M$ to model and predict
  - Assumption of M many independent features
  - Assumption of data points have order independence
    - A shuffle would not change the performance
- The knowledge can be carried by a sequence of data points $\langle x_t \rangle$
  - Speech, video, events on a network
  - Then the learning method is called sequence learning
- A sequence-to-sequence model, $\mathbf{v}$ is encoded input vector

$$\mathrm{P}(y_1 \dots y_{T0} \mid x_1 \dots x_T) = \prod_{t=1}^{T0} \mathrm{P}(y_t \mid \mathbf{v}, y_1 \dots y_{t-1})$$

Autoregressive Models

- Models predict the next data point in a sequence based on the previous ones
  - The next word in a sentence, or pixel in an image is predicted via prior elements
- Generate outputs sequentially, every output conditioned on the previous outputs
- Maximize likelihood by predicting future values based on past data
  - Autoregressive moving average models
  - Recurrent neural networks
  - Generative pre-trained transformer (GPT)
- Generation might be slower due to happening sequentially
- Hard to capture long-term dependencies (Why?)
  - This is mitigated by **Attention** neural networks and Transformers

Autoregressive vs Latent Space Models

Sequential Non-Sequential	Efficiency	Interpretability	Training Complexity
▪ Autoregressive models are naturally **sequential** ▪ Latent space models can generate data in **parallel**	▪ Autoregressive models can be **slower** ▪ Latent space models are **faster**	▪ Autoregressive models are **easier to interpret** since each step depends on the previous ones ▪ Latent space models are less interpretable due to the **abstract** nature of latent representations	▪ Autoregressive models optimize likelihood at each step ▪ Latent space models, especially GANs, can be more **difficult** to train and are prone to stability issues

Holt-Winters Model

- Late 1950s, basic exponential smoothing proposed
  - One of the first time-series forecasting methods

$$y_t = \alpha x_t + (1-\alpha)y_{t-1},\ 0 < t,\ 0 \le \alpha \le 1$$

  - $\langle x_t \rangle$ is the raw data sequence to be smoothed, $\langle y_t \rangle$ is the output (estimate)
  - Calculate y recursively (y_{t-1} is the most recent forecast) beginning with $y_0 = x_0$

$$y_t = \alpha x_t + (1-\alpha)x_{t-1}\ (1-\alpha)^2 x_{t-2} + \cdots + (1-\alpha)^{t-1}x_1 + (1-\alpha)^t\ x_0$$

- Optimize to determine the operational value of α

Root Mean Squared Error **RMSE** $= \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_t - x_i)^2}$

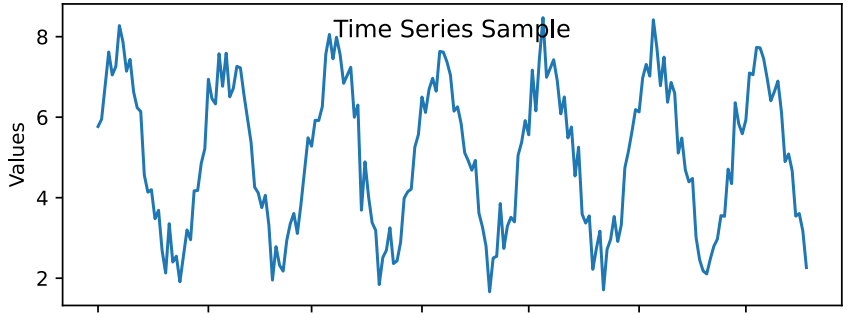
Mean Absolute Relative Error **MARE** $= \sum_{i=1}^{n}\left|\frac{y_t - x_i}{y_t + 1}\right|$

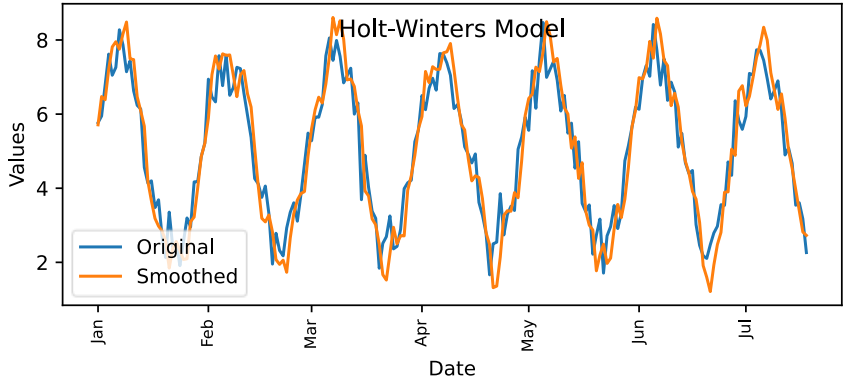
Holt-Winters Example

- 7 seasons
- Smoothed model can be used to forecast

```
from statsmodels.tsa.holtwinters
import ExponentialSmoothing

model = ExponentialSmoothing(
  hwdata,
  seasonal='add',
  trend='add',
  seasonal_periods=7)
ypred = model.fit()
# ypred.fittedvalues
```





ARIMA

- Autoregressive Integrated Moving Average models combine two time series modeling
  - Auto-Regressive (AR)
  - Moving Average (MA) models
- The AR model is constructed from the given series $\langle x_t \rangle$ so that the modeled y depends only on its lags
- The MA model is constructed so that y depends on the lagged forecast errors
- Non-seasonal ARIMA(p,d,q) models are comprised of three terms p, d, and q
- **p** order of the AR term (the number of lags of y that are used as predictors)
- **d** minimum number of difference-ing needed to make the time series stationary
- **q** order of the MA term (number of lagged forecast errors should go into the model)

ARIMA Model

- The autoregressive model of order p is given by

  $$y_t = c + \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + \varepsilon_t$$

- The moving average model of order q is given by

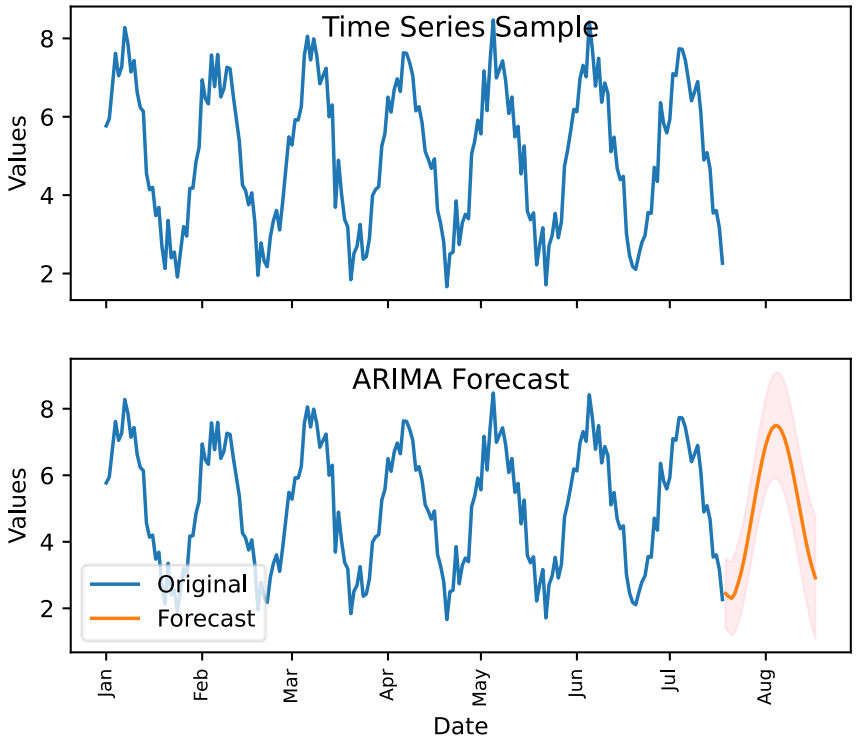
  $$y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

- where
  - ε_t error, white noise, at time t
  - Mean, $\mathrm{E}[\varepsilon_t] = 0$
  - Variance, $\mathrm{Var}(\varepsilon_t) = \sigma^2$
  - ε_{t_i} are independent

ARIMA Example

```
import pmdarima as pm

model = pm.auto_arima(arimadata,
  seasonal=False,
  error_action='ignore',
  suppress_warnings=True)
# Forecast the next 30 periods
  (e.g., 30 days ahead)
n_periods = 30
forecast, conf_int =
  model.predict(n_periods=n_periods,
  return_conf_int=True)
```



ARIMA Methodology

- Check Stationarity
  - If not stationary, differencing is applied until it becomes stationary
  - First-order differencing

$$y_t' = y_t - y_{t-1}$$

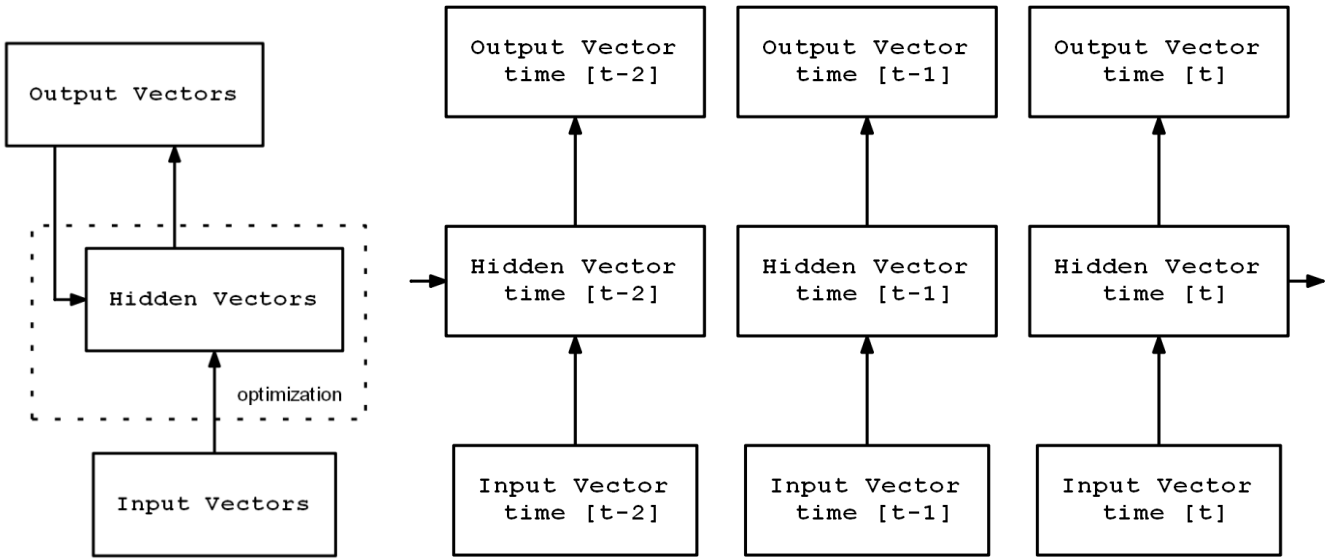
- Determine parameters p, d, and q
  - Analyze (partial) autocorrelation plots
- Fit model using MLE
- Forecast
- Evaluate model
  - RMSE or MAE on a validation dataset
- Increasing p may capture more complex patterns at the risk of overfitting
- Increasing d can help in making the data stationary at the risk of information loss
- Increasing q allows the model to capture more short-term fluctuations at the risk of noise and overfitting

Recurrent Neural Networks

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Recurrent Neural Networks

- The representation of a seq is learned by maintaining a hidden state vector capturing the current state of the seq
- The hidden state vector is computed from both the current input vector and the preceding hidden state vector
- Rightmost "unrolled" view of the RNN



Sequence-to-Sequence Models

- A set of N many data points $x_i \in \mathbb{R}^{\mathrm{M}}$ in M dimensions are assumed to have independent features with no dependence on their order (i.e. a shuffle is OK)
- When the knowledge is carried by a sequence of data points $\langle x_t \rangle$, such as speech, video, text, events on a network, etc., then we deal with a timeseries or sequence
- Sequences can be represented naturally on the directed graphs of RNNs
- **Encoder-Decoder Framework** The input $\mathbf{X}$ is used to generate the hidden states

  $$h_t = f_{enc}(x_t, h_{t-1}; \theta_{enc})$$

  by the encoder E can be used by the decoder D output states and context vector

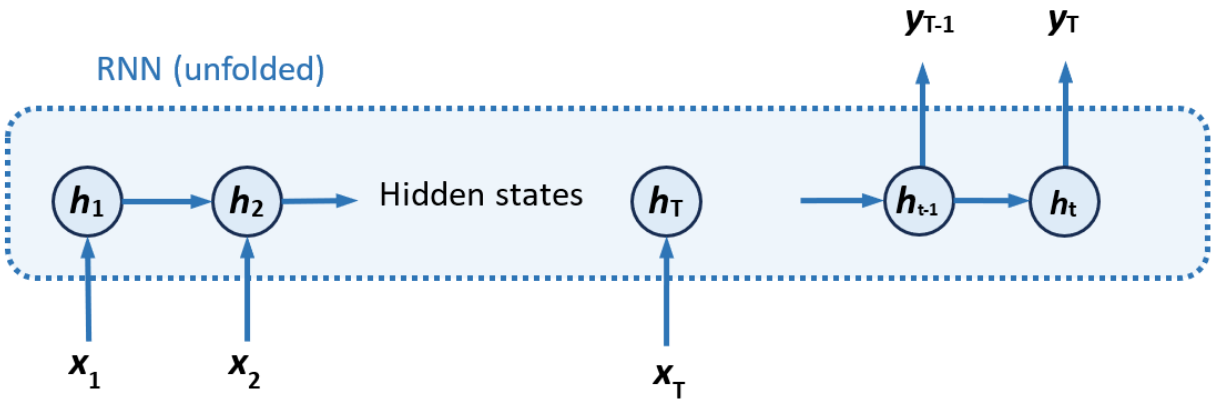
  $$s_t = f_{dec}(y_{t-1}, s_{t-1}, c_t; \theta_{dec})$$

  to generate an output sequence $\mathbf{Y}$, where the context vector is given by

  $$c_t = \sum_{i=1}^{T} \alpha_{t,i} h_i$$

RNN and Seq-to-Seq Model

- RNN can have multiple input-output combinations depending on the output(s) connected to the hidden layers (i.e., state/memory/thought layers)
  - such as one-to-one, one-to-many, many-to-one, and many-to-many
- We use indirect many-to-many or sequence-to-sequence (seq2seq), where an input sequence is transformed into another, different output sequence, not necessarily with the same length as the input



Long Short-Term Memory RNN

- In practice, RNNs are susceptible to the vanishing gradient problem, or exploding gradient due to the nature of the feedback network
- LSTM variant has three layers for control: input, output, forget gates

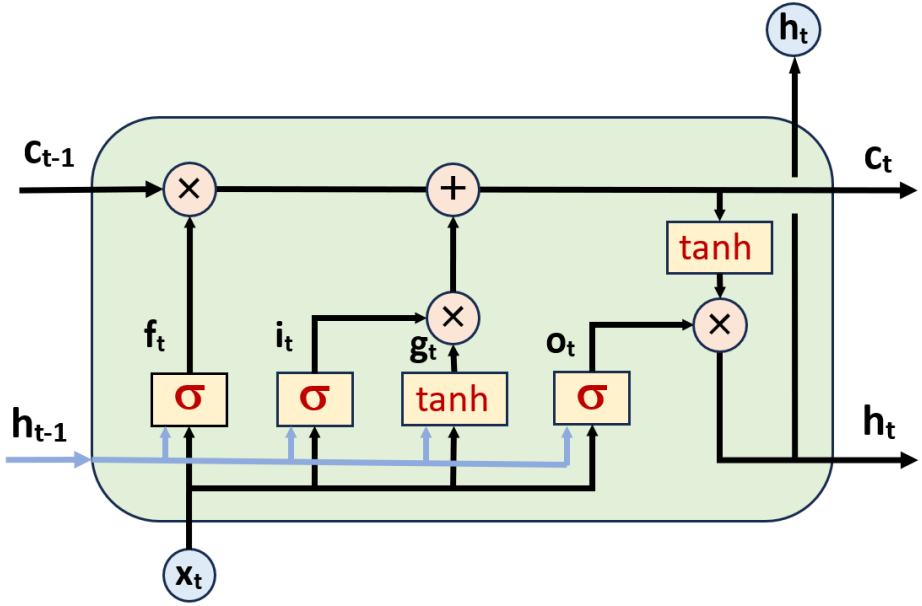
$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi})$$

$$g_t = \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg})$$

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho})$$

$$f_t = \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf})$$

- Cell state $c_t = f_t \odot c_{t-1} + i_t \odot g_t$
- Hidden state $h_t = o_t \odot \tanh c_t$



Example RNN

```
class PyTorchRNN(nn.Module):
  def __init__(self, n_hidden=128, numH=2,
epochs=500, eta=0.005, B=1920, w=None,
dropout=0.0):
    super().__init__()
    self.rnn, self.outlayer = None, None
    self.softmax = nn.LogSoftmax(dim=1) #
loss function, last layer is nn.LogSoftmax
    self.loss_func = nn.NLLLoss(weight=w)

  def forward(self, _X):
    embeds = self.letter_embeddings(_X)
    lstm_out, _ = self.rnn(embeds) # output
is batched
    # Use last hid layer outp - W x Seq[T]
    output = self.outlayer(lstm_out[:, -1,
:]) # Use only the last hidden layer - Hn
    output = self.softmax(output)
    return output
```

```
  def predict(self, _X):
    with torch.no_grad():
      output = self.forward(_X)
    p_values, indices = output.max(dim=1)
    return indices.to('cpu')

  def fit(self, _X_train, _y_train,
info=False, **kwargs):
    self.letter_embeddings =
nn.Embedding(M, EMBEDDING_M, device=Device)
    self.rnn = nn.LSTM(EMBEDDING_M,
self.n_hidden, self.numH,
bidirectional=False, bias=False,
batch_first=True, dropout=self.dropout,
device=Device)
    self.outlayer =
nn.Linear(self.n_hidden, C, device=Device)
    self.optimizer =
optim.Adam(self.parameters(), lr=self.eta)
```

References

- Foster, David. Generative deep learning. " O'Reilly Media, Inc.", 2023.
- Bishop, Christopher M., and Nasser M. Nasrabadi. Pattern recognition and machine learning. Vol. 4. No. 4. New York: springer, 2006.
- Goldberg, David E. "Optimization, and machine learning." Genetic algorithms in Search (1989).
- Fukunaga, Keinosuke. Introduction to statistical pattern recognition. Elsevier, 2013.
- Kuhn, M. "Applied predictive modeling." (2013).
- Mitchell, Melanie. An introduction to genetic algorithms. MIT press, 1998.
- Theodoridis, Sergios, and Konstantinos Koutroumbas. Pattern recognition. Elsevier, 2006.

JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.